

Algoritmo Backtracking

Sudoku

1. Prints do código

```
1 class Main {
2
3     // Função que verifica se é seguro colocar um número na posição [row][col]
4     static boolean isSafe(int[][] mat, int row, int col, int num) { 1 usage
5
6         for (int x = 0; x < 9; x++) // Verifica se já há o num na linha
7             if (mat[row][x] == num)
8                 return false;
9
10        for (int x = 0; x < 9; x++) // Verifica se já há o num na coluna
11            if (mat[x][col] == num)
12                return false;
13
14        int startRow = row - (row % 3); // Verifica se já existe numero no bloco 3x3
15        int startCol = col - (col % 3);
16
17        for (int i = 0; i < 3; i++)
18            for (int j = 0; j < 3; j++)
19                if (mat[i + startRow][j + startCol] == num)
20                    return false;
21
22        return true; // Se passou em todas as verificações, é válido
23    }
24
25    static boolean solveSudokuRec(int[][] mat, int row, int col) { // Função RECURSIVA principal 3 usages
26
27        if (row == 8 && col == 9) // CONDIÇÃO DE PARADA:
28            return true; // Se chegou ao final da matriz, solução encontrada
29
30        if (col == 9) { // Se chegou ao fim da coluna, vai para a próxima linha
31            row++;
32            col = 0;
33        }
34
35        if (mat[row][col] != 0) // Se a célula possui um valor que seja diferente de 0, vai para a próxima
36            return solveSudokuRec(mat, row, col: col + 1);
37
38        for (int num = 1; num <= 9; num++) { // Teste de números de 1-9
39
40            // Verifica se a escolha é válida
41            if (isSafe(mat, row, col, num)) {
42
43                //insere o numero e avança recursivamente
44                mat[row][col] = num;
45
46                // Se a recursão a partir daqui tiver resultado, retorna true
47                if (solveSudokuRec(mat, row, col: col + 1))
48                    return true;
49
50                /* Caso a solução não levou a um resultado válido o BACKTRACKING (desfaz a escolha)
51                 o for testa um outro numero*/
52                mat[row][col] = 0;
53            }
54        }
55
56        return false; // Caso nenhum não funcione, retorna false
57    }
58 }
```

```

1  class Main {
59  // Função principal que inicia o processo
60  static void solveSudoku(int[][] mat) { // 1 usage
61      solveSudokuRec(mat, row: 0, col: 0);
62  }
63
64  public static void main(String[] args) {
65
66      int[][] mat = {
67          {3, 0, 6, 5, 0, 8, 4, 0, 0}, // Matriz representando o tabuleiro inicial
68          {5, 2, 0, 0, 0, 0, 0, 0, 0}, // 0 representa a célula vazia
69          {0, 8, 7, 0, 0, 0, 0, 3, 1},
70          {0, 0, 3, 0, 1, 0, 0, 8, 0},
71          {9, 0, 0, 8, 6, 3, 0, 0, 5},
72          {0, 5, 0, 0, 9, 0, 6, 0, 0},
73          {1, 3, 0, 0, 0, 0, 2, 5, 0},
74          {0, 0, 0, 0, 0, 0, 0, 7, 4},
75          {0, 0, 5, 2, 0, 6, 3, 0, 0}
76      };
77
78      solveSudoku(mat); // resolve o sudoku
79
80      for (int i = 0; i < mat.length; i++) { // Imprime o tabuleiro final
81          for (int j = 0; j < mat[i].length; j++)
82              System.out.print(mat[i][j] + " ");
83          System.out.println();
84      }
85  }
86  }
87
88

```

2. Explicação

O algoritmo percorre o tabuleiro célula por célula por vez, da esquerda para a direita e de cima para baixo, ao encontrar uma célula vazia que é representada por '0' testa os números de 1 a 9 em ordem, se o número for válido ele avança para a próxima célula.

Em algum momento em que nenhum número de 1 a 9 puder ser inserido em uma célula vazia, o algoritmo retrocede, desfazendo a última inserção e assim zerando a célula, e tenta o próximo número do nível anterior da recursão. Esse processo tentativa e retrocesso continua até que o tabuleiro seja totalmente preenchido

2.1 Soluções Parciais

A cada chamada recursiva o algoritmo trata o estado atual do tabuleiro como uma solução parcial. A inserção de um número válido representa um avanço onde o estado parcial é armazenado na própria matriz.

2.2 Teste de possibilidade

Antes de inserir um número em uma célula, a função **isSafe()** vai verificar três restrições simultaneamente:

- 1- O número já não pode existir na mesma linha;
- 2- O número já não pode existir na mesma coluna;
- 3- O número já não pode existir na submatriz 3x3 no qual a célula pertence

Só se essas 3 verificações passarem, o número é considerado seguro.

2.3 Retrocesso

O retrocesso acontece quando a chamada recursiva retorna *false*. Nesse momento a instrução **mat[row][col] = 0** desfaz a última inserção, colocando a célula no modo vazio novamente e então o loop testa outro número, caso todos os números sejam testados a função retornará *false*. Assim retornando para o nível anterior.

2.4 Solução válida

A recursão termina quando o algoritmo tenta avançar além da última célula do tabuleiro, quando **row == 8 && col == 9**, indicando que todas as células foram preenchidas corretamente

3. Estruturas de dados

- Matriz bidimensional: **int [][]** usada para montar a estrutura do tabuleiro. Cada elemento de **mat[i][j]** corresponde a célula da linha i e coluna j
- Pilha implícita de recursão (memória do backtracking): Como o backtracking depende de pilha de chamadas da recursão, o JVM mantém uma pilha de ativação para cada chamada recursiva. Cada frame da pilha armazena os valores dos parâmetros row e col, além também do endereço de retorno para a instrução anterior

4. Entrada e Saída do tabuleiro

Entrada:

3	0	6	5	0	8	4	0	0
5	2	0	0	0	0	0	0	0
0	8	7	0	0	0	0	3	1
0	0	3	0	1	0	0	8	0
9	0	0	8	6	3	0	0	5
0	5	0	0	9	0	6	0	0
1	3	0	0	0	0	2	5	0
0	0	0	0	0	0	0	7	4
0	0	5	2	0	6	3	0	0

Saída:

3	1	6	5	7	8	4	9	2
5	2	9	1	3	4	7	6	8
4	8	7	6	2	9	5	3	1
2	6	3	4	1	5	9	8	7
9	7	4	8	6	3	1	2	5
8	5	1	7	9	2	6	4	3
1	3	8	9	4	7	2	5	6
6	9	2	3	5	1	8	7	4
7	4	5	2	8	6	3	1	9

O algoritmo inicia na célula (0,1), a primeira célula vazia. Testa o número 1: verifica linha 0 (não há 1), coluna 1 (não há 1), submatriz superior esquerda (não há 1). O número é válido e inserido. A recursão avança para (0,4). Esse processo se repete até que uma contradição force o retrocesso. No pior caso, o algoritmo pode retroceder várias vezes antes de convergir para a solução exibida acima (saída).

5. Referência

GEEKSFORGEEEKS. *Sudoku Backtracking*. Disponível em:

<https://www.geeksforgeeks.org/sudoku-backtracking-7/>. Acesso em: abr. 2026